

BMB 306

YAZILIM MÜHENDİSLİĞİ



Gerçekleme Aşaması

- ❧ Gerçekleme aşaması kodun yazılmaya başlandığı aşamadır.
- ❧ Tasarımın belli bir seviyeye gelmiş olması gereklidir.
- ❧ Gerçekleme aşamasında büyük risklerden biri izlenebilirliğin iyi olmamasıdır. Bu aşamada herhangi bir anda gerçeklemenin ne kadarının bittiğini ve geriye ne kadarlık iş yükünün kaldığını belirlemek zor ve yapılması da o kadar önemli bir iştir.
- ❧ Bu aşamada herhangi bir anda ne durumda olduğunuzu net olarak bilerseniz, proje süresi hakkında çok gerçekçi tahminlerde bulunabilir ve riskleri aşabilirsiniz.

Gerçekleme Aşaması

🌀 Hiçbir Şey Yapmayan Bir Kod Yazın

- Bir projede sistem üzerinde koşturulabilen bir programı görmek (program çok az iş yapsa ve hatalarla dolu olsa bile) gerek moral motivasyon gerekse büyük resmi anlamak açısından çok faydalıdır.
- Kodlamaya başladığınızda ilk işiniz bir iskelet oluşturmak olsun.
- Derleme ve bağlama işlemlerinizi geçtiğinden emin olduğunuz bu iskelet üzerinden kodlamaya devam edin.
- Daha sonra henüz içleri boş olan ana işlevlerinizi kodlayın. Bu anda da kodunuzun derleme ve bağlama işlemlerinde takılmadığını gösterin.
- İçleri boş da olsa ana işlevleri kodlamak tasarımınızın sağlamlığını da teyit eder. Henüz hiçbir iş yapmayan bu kodunuzu mümkünse çalıştırın ve işlevlerin birbirini sağlıklı bir şekilde çağırdığından emin olun.
- Bu aşamada hiçbir iş yapmayan işlevlerin içine ekrana bilgi verici satırlar döken satırlar ekleyebilirsiniz. Böylece kod akışını tespit edebilirsiniz.

Gerçekleme Aşaması

🌀 Hiçbir Şey Yapmayan Bir Kod Yazın (devam)

- Özellikle çok kodlayıcılı sistemlerde iskelet oluşturarak kodlamanın çok faydası vardır.
- Öncelikle motivasyon faydası vardır; çok az iş yapsa bile sonuçta birşeylerin çalıştığını görmek ekip elemanlarının motivasyonunu yükseltir.
- İkinci sebep de belli bir aşamadan sonra ekipteki her kodlayıcı kendi kodunu bağımsız olarak kodlayabilecek ve test edebilecektir.
- Entegrasyon genelde çok zaman alıcı ve üstelik de planlara konulması unutilan veya ihmal edilen bir safhadır. En başta iskelet oluşturup bu iskelet sürekli derlenebilir ve çalıştırılabilir durumda tutulduğunda entegrasyon süresinden de bayağı tasarruf edilir.
- Unutmayın ki tüm kod bir seferde yazılamaz. Programlar canlı organizmalar gibi büyür ve gelişirler. Hiç kimsenin büyük bir adam/kadın gibi dünyaya gelemeyeceği gibi programlar da önce bebek olarak dünyaya gelmek zorundadır.

Gerçekleme Aşaması

∞ Yazılım Sürüm Tutma Gereçleri – Konfigürasyon Yönetimi

- Mutlaka bir sürüm tutma yazılımı kullanın.
- Sürüm tutma yazılımları 3 ve daha fazla kişinin bulunduğu projelerde kritik öneme sahiptir ve kullanılmaları kaçınılmazdır.
- Ancak tek başına bile çalışıyorsanız bir sürüm tutma yazılımı kullanmanızı tavsiye ederim.

Gerçekleme Aşaması

Yazılımınızı Tek Bir Yerde Saklamayın

- Bilgisayarım çökecek mi diye düşünmeyin. Evet mutlaka çökecek. Yazılımınızı mümkün olan en kısa periyotta yedekleyin.
- Modern editörlerin çoğu otomatik kaydetme (Autosave) özelliğine sahiptir. Bu özelliği mutlaka açık duruma getirin ve örneğin her 10 dakika gibi bir saklama aralığı belirleyin.

Gerçekleme Aşaması

🌀 İyi Kod Basit Olandır (KISS Prensibi – Keep It Simple)

- Kodu yeni mezun olmuş birine verin. Eğer birkaç gün içinde adapte olabiliyor ve kodu değiştirebiliyorsa kodunuz iyi bir koddur, eğer kodu değiştirmek için uzman bekliyorsanız iyi bir kod değildir.

"Zarafet (Basitlik, sadelik, açıklık) çöpe atılabilir bir lüks değil, aksine başarı ve başarısızlık arasındaki seçimi belirleyen bir özelliktir."

Edsger W. Dijkstra – Dijkstra Algoritması'nın yaratıcısı

"Eğer bir programın genel yapısını düşünürken tüm özülle kavrayamıyorsanız o programı kodlamaya hazır değilsiniz demektir."

Richard Pattis - University of California, Irvine

"Yazılım tasarımı kurabilmek için iki yöntem vardır: Birinci yöntemde tasarım o kadar basit yapılır ki ortada hiçbir eksiklik kalmaz, ve diğer yöntemde tasarım o kadar karmaşık yapılır ki ortada hiçbir eksik fark edilmez. Elbette ilk yöntem daha zordur."

C.A.R. Hoare – Quicksort'un yaratıcısı

Gerçekleme Aşaması

∞ İyi Kod Anlaşılır Olandır

- Anlaşılır kod başkaları tarafından okunacağı düşünülerek yazılmış olan koddur.
- Anlaşılır olması için mucizelere gerek yoktur; paragraf girintilerinin düzenli olması, değişkenlerin iyi adlandırılmış olması, işlevlerin (fonksiyonların) sadece tek işi yapması ve yine iyi adlandırılmış olmaları vb. konulara dikkat etmek bile anlaşılabilirliği gayet arttırır.

"Cinayet gizemlerini çözmeye "tamam" denilebilir ancak bir yazılım kodunu çözmek ihtiyacını duymamalısınız. Yani kodu basitçe okuyabilmelisiniz."

Steve McConnell – "Code Complete" Kitabının Yazarı

Gerçekleme Aşaması

∞ İyi Kod Değiştirilmesi ve Bakımı Kolay Olan Koddur

- Değiştirilmesi kolay ve güvenli kod "bakım yapılabilir" koddur.
- Projedeki en büyük maliyet öğelerinden biri bakımdır. Bakım yapılabilir kod anlaşılır, bilgi verici, değişikliklere uyum sağlayabilir ve hata ayıklaması kolay olan koddur."
- Kodun bakım yapılabilir ve sürdürülebilir olması için şunlara dikkat edilmelidir:
 - Test kodu yazın
 - Modüler tasarım yapın
 - Kodlama standartları uygulayın
 - Uygun isimler kullanın
 - Sürekli-iyileştirme (Refactoring) yapın
 - Kod tekrarını önleyin
 - Mümkünse okunaklılığı performansa tercih edin
 - Uzun metotlardan kaçınin
 - Standart algoritmalar ve haberleşme protokolleri kullanın
 - Ne kadar zeki olduğunuzu kodda göstermeye çalışmayın
 - Projenizi dokümante edin ve iyi yorumlar yazın

Gerçekleme Aşaması

İyi bir Kod Veri Yapıları İyi Kurulmuş Olandır

- Yazılıma yeni başlayanların yaptıkları ihmallerden biri de veri yapılarına gereken önemi vermemek ve onları yeterince öğrenmemektir.
- Genellikle kısa bir sürede karmaşık algoritmaların kodlaması başarılabilirken, veri yapılarını özümsemek ve beyinde sağlam bir yere oturtmak yılları almaktadır.
- Usta tasarımcılar genelde çok kaliteli algoritmalar bilen insanlar değildir, aksine üniversiteden yeni mezun olmuş biri hesaplama ve algoritma kurmada daha başarılı olabilir, ancak iş veri yapılarını kurmaya geldiğinde gerçekten de bilgi birikimi ve tecrübe ön plana çıkmaktadır.
- Veri yapıları en az algoritmalar kadar önemlidir.

"İyi veri yapıları ve kötü kod, bu ilişkinin tam ters olduğu durumdan çok daha iyi çalışır."

Eric S. Raymond – Açık Kaynak Yazılımın Öncülerinden

"İddiam şu ki iyi programcı ve kötü programcı arasındaki fark o kişinin kodunu mu yoksa veri yapılarını mı daha önemli gördüğüne bağlıdır. Kötü programcılar kodları için kaygılanırlar. İyi programcılar ise veri yapıları ve onlar arasındaki ilişkiler için kaygı duyarlar."

Linus Torvalds

Gerçekleme Aşaması

❧ İkinci Kodlama Sürümü İle İlgili Garip Olay (Second System Effect – Fred Brooks)

- Ürettiğiniz ufak, amaca uygun, hızlı çalışan birinci sistemi veya yazılım sürümünü başarı ile teslim ettiniz.
- Ardından aynı sistemin daha gelişmiş olan ikinci sürümü üretmek için işe koyuldunuz ancak ikinci sisteminiz kocaman hantal bir su aygırı olarak çıktı 😊
- Yeni sürümde birçok yeni özellik koydunuz ancak bu özellikler sistemi yavaş ve hantal hale getirdi. Bu ikinci kez düzenlenmiş sistemlerde veya kodlama sürümlerinde sık rastlanan bir olaydır.
- İnsanlar bu benim başıma gelmez diye düşünür ama dikkat etmezseniz başınıza kolayca gelir. – (Brooks, 1995)
- İkinci sürümün garip çıkmasının birçok sebebi vardır. Bir sebebi birinci sürümün başarısının getirdiği aşırı kendine güvendir. Birinci sistemde eksik kalan özellikleri koyma çabası ile hantal, kullanışsız ve özellik çorbası bir sistem üretmek işten bile değildir.
- Aslında birinci sistemi yaparken gerçekten gerekli ve performansı bozmayacak özellikleri sisteme koyduğunuz halde ikinci sistemde çok da gerekmeyecek, karmaşık özellikleri de sisteme koyma hatasına düşölür.
- Diğer sebep de birinci sistemdeki başarının iyice irdelenmemesidir. Birinci sistem neden başarılıdır? Örneğin birinci sistemde fazla özellik yoktur ve kullanması kolaydır. Örneğin sistem kaynakları sisteme tam yetmiştir ve bu yüzden sistem performansı en tepededir. Oysa şimdi ikinci sistemi yaparken bu koşulları bozduğumuzun farkında olmak gerekir.

Gerçekleme Aşaması

- 🌀 **Bu da Olsun, Şu da Olsun (Feature Creep (Özellik Yığılması))**
- Bir programı bir ev gibi düşünecek olursak, yani benzetme yaparsak:
 - **Başlangıç ve ana ihtiyaçlar aşaması** – Eve ilk taşındığımızda birçok eksik vardır. Evi temel ihtiyaçları görecektir şekilde donatırsınız.
 - **Olgunluk aşaması** – Evdeki ana eksikleri gidermiş ve sizin için gerekli diğer ihtiyaçları da sağlamış durumdasınızdır, ve hatta hayatı kolaylaştıracak bir iki lüksünüz de vardır.
 - **Dolup taşma aşaması** – Ev dolup taşmaktadır, bir şeyi yapmak için her gereçten iki tane vardır, herkes bir şeyler hediye etmiştir ve gerçekten hangisini kullanacağım diye kafanız karışmaktadır, evde hareket etmek iyice zorlaşmıştır, zaman zaman kazalar meydana gelmektedir. Bir şeyleri atmak istemektesinizdir ama birilerinden çekindiğiniz için atamamaktasınız.

Gerçekleme Aşaması

∞ Bu da Olsun, Şu da Olsun (Feature Creep (Özellik Yığılması)) - devam

- İşte "Bu da olsun, şu da olsun" ya da İngilizce adı ile "Feature Creep" yukarıdaki örnekteki gibi dolup taşma aşaması gibidir. Program birçok gereksiz özellik ile dolup taşmakta, iyice yavaşlamış durumdadır. Kullanıcılar menülerde kaybolmaktadır. Bazı menüler eski mantıkla, bazıları yeni mantıkla yazılmıştır ve bir işi yapmak için birçok menü seçeneği vardır. İnsanların kafası iyice karışmıştır. Bakım maliyeti zorlaşmıştır. En kötüsü de geliştirme süresi kadar bir süre de bu ıvır zıvır ve gereksiz özellikleri gerçeklemek için harcanmış ve maliyet kat kat artmıştır.
- Bu olay birçok projeyi başarısızlığa uğratan ana etkenlerden biridir. Sağlıklı çalışan bir programınız varsa ona yeni eklemeler yapmadan önce iki kez düşünün. Kullanıcıların kullanma alışkanlıklarından programın performans kriterlerine kadar birçok konuyu gözden geçirin. Bu özellik gerçekten gerekli mi diye kendinize mutlaka sorun. Eğer bütün bunlara cevabınız olumlu ise ancak bu durumda özelliği gerçekleyin. Özellikle de kimsenin kullanmayacağı özellikleri programınıza eklemekten kararlı bir biçimde kaçınin.

"Tasarımda mükemmelliğe, daha fazla ekleyecek özellik kalmadığında değil, daha fazla atılacak şey kalmadığında ulaşılır."

Antoine de Saint-Exupéry, "Küçük Prensin" Yazarı

Gerçekleme Aşaması

☞ Scope Creep (Hedef Yığılması)

- Literatürde birbirine çok yakın iki kavram vardır: Scope Creep (Hedef Yığılması) ve Feature Creep (Özellik Yığılması).
- Eğer müşteri istekleri ve gereksinimleri sürekli değişiyor ve kontrolsüzce artıyorsa buna Hedef Yığılması adı verilir.
- Eğer proje takımı durmadan aslında pek de gerekli olmayan özellikleri yazılıma ekleyip duruyorsa buna Özellik Yığılması adı verilir.
- Sonuçta her ikisi de şişmiş bir yazılıma sebep olur. Bunların sonucunda sistemde performans sorunları oluşur, karmaşık ve kullanması zor bir sistem üretilmiş olur. Bütçe ve zaman aşımı oluşur. Kalite düşer. Rünün bakımı zorlaşır.

"Karmaşıklığın temel bir nedeni şudur ki üreticiler kullanıcıların istediği hemen her özelliği fazla incelemeden, üzerinde düşünmeden benimseyip gerçekleştirmektedirler."

Niklaus Wirth – Pascal'ın Yaratıcısı

Gerçekleme Aşaması

Altın Harflerle Yazmaya Çalışmak (Gold Plating)

- Özellik yığılmasına çok yakın diğer bir kavram da "Altın Harflerle yazmaya çalışmak" kavramıdır.
- Aşırı mükemmeliyetçi bir yaklaşımdır.
- Gerçekleştiricilerin mükemmel bir sistem üretme amacına saplanıp detaylarda boğulması ile sonuçlanır.
- Proje süreleri aksar, sistemin bir bölümü mükemmel iken diğer kısımlarının üretimine hiç başlanmamış olabilir.

Gerçekleme Aşaması

∞ Hata Bildirim Sistemi Kullanın

- Öncelikle bir hata bildirim sistemi (Bug Reporting System) kurun.
- Bu hata bildirim sistemi kağıtlarla yürüyen bir sistem olabileceği gibi daha da iyisi bir program aracılığıyla işleyen bir sistem olmalıdır.
- Bu sistemde hata adı, hatanın detayları, oluşma koşulları, hatayı kimin bildirdiği, hatanın düzeltim durumu, hatayı kimin düzelittiği, hatayı kimin düzelmiş olarak kabul ettiği gibi bilgiler bulunmalıdır. En tercih edilen durum hatayı açan kişinin kapatan kişi olmasıdır.

Gerçekleme Aşaması

🌀 Birim Testleri Yazmayı Alışkanlık Haline Getirin

- Birimler kullanıcıların mantıklı olarak bölebileceği ve tek bir işi yapan en küçük parçalardır. Bu birim gerçekten tek bir işi yapan bir işlev (fonksiyon) olabileceği gibi bir iş grubunu içeren bir dinamik bağlı kütüphane (DLL) de olabilir. Birim testi sadece bir birimi test etmek için yazılmış olan testtir.
- İdealde birim testleri ilgili test edilecek birimden daha önce yazılmalıdır. Evet yanlış duymadınız test kodu daha önce yazılmalıdır. Eğer buna zaman bulamazsanız birim testleri kodla aynı zamanda yazılmalıdır. Birim testleri koddan sonra yazsanız dahi hiç yazmamaktan iyidir.
- Birim test kodu, ilgili ürün kodunun içerisinde yer almaz. Ayrı dosyalarda, ayrı modüllerde tutulurlar.
- Birim testlerinin tümünün başarılı olarak geçmesi yazılımın bütünüünün başarılı olduğu anlamına gelmez. Bütünün başarı oranı entegrasyon, kurulum ve saha testlerinde ortaya çıkacaktır.
- Birim testler özellikle gerileme kontrolünde (yani yeni kod eklediğimizde başka birşeyi bozmuş muyuz kontrolünde) çok faydalıdır. Özellikle otomatik birim testleri size günler kazandırır ve piyasaya bozuk ürün sunup müşterinin size olan saygısını yitirmenize engel olur.

Gerçekleme Aşaması

🌀 Birim Testleri Yazmayı Alışkanlık Haline Getirin (devam)

- Birim testleri test aşamasında değil de gerçekleştirme aşamasında incelememizin sebebi birim testlerin gerçekleştirme aşamasının en başından başlanıp yazılması gereken testler olmasıdır.
- Birim testlerin dezavantajı test yazmak için de zaman harcanacak olmasıdır, ancak bu zaman daha sonra birim testlerin sağlayacağı faydalar ile telafi edilebilir. Birim testler çıkabilecek hata sayısını çok büyük oranda azaltır.
- Diğer zorluk ise tüm ekibin birim test yazmaya başlamasıdır. Bir kişi bile direnç gösterse onun yazdığı kısımlar birim testlerden geçmediği için eksik kalır. Örnek verecek olursak herhangi bir modülü birim testlerinde ihmal etmek, fren borularından sadece bir bozuk araba ile yola çıkmaya benzer.
- Birim testleri yazmak her zaman uygun yada yapılabilir olmayabilir. Örneğin kullanıcı arabirimi (GUI) için birim test yazmak çoğu zaman zor hatta imkansızdır.
- Birim testler her zaman günce tutulmalıdır. Birim testler de yazılımın bir parçası imiş gibi onları korumalı, güncellemeli ve bakımını yapmalısınız; bu da kuşkusuz ekstra maliyettir. Ancak tüm maliyete karşın birim testler size çok yarar sağlayacaktır.

Gerçekleme Aşaması

🌀 Kod Gözden Geçirme Toplantısı

- Kod gözden geçirme toplantıları sisteminizde oluşacak hataları önceden engellemenizi sağlar. Çok yararlı olan kod gözden geçirme toplantılarının kullanımı önerilmektedir.
- Kod gözden geçirme toplantısında bulunacak kişiler ve görevlerini listeleyecek olursak:
 - Kodun kodlayıcısı,
 - Moderatör (Toplantının yöneticisi)
 - Okuyucu (Kodu veya dokümanları adım adım okumak veya izleyebilmek için)
 - Yazıcı (İşlem maddelerini ve önerilen düzeltmeleri/iyileştirmeleri not etmek için)
 - Yorumcular (Her seviyeden yorum veren yazılımcılar ve öğrenim için katılan yeni yazılımcılar)

"Gözden geçirme işlemleri üretkenliği, kaliteyi ve proje kararlılığını (stabilitesini) büyük ölçüde arttırır."

Michael E. Fagan – Yazılım Test Uzmanı

Gerçekleme Aşaması

☞ Mentorluk (Abilik, Ablalık)

- Genellikle yeni işe başlayan yazılımcılar bir iki kursa gönderildikten sonra ve bazen hiç kursa bile gönderilmeden onlardan artık iş çıkarmaları beklenir. Oysa örneğin bir iki kursa gitmiş bir işletmeciyi şirketinizin mali hesaplarını yönettirir misiniz, veya bir iki kursa gitmiş bir avukata kendinizi savunma görevini verir misiniz? Tabii ki hayır. Yazılım işi de bu kadar ciddiye alınması gereken bir iştir. İyi bir yazılımcının yetişmesi yıllar alır.
- İlk eğitimden sonra yazılımcı daha tecrübeli bir yazılımcı ile birlikte ve hatta onun gözetiminde çalıştırılmalıdır. Yeni yazılımcının birlikte çalışacağı mentor iletişim kabiliyeti yüksek, teknik olarak sağlam ve mentorluk tecrübesi olan bir kişi olmalıdır.
- Ayrıca mentor sadece eğitimci değil projelerde aktif görev alan, proje detaylarını bilen bir insan olmalıdır. Herhangi bir kişi mentorluk yapamaz, bu özel yetenekler gerektirir.
- Mentor başına düşen yeni yazılımcı sayısı da fazla olmamalıdır.

Test ve Birleştirme (Entegrasyon) Aşaması

- ✎ Test süreci en az kodlama süreci kadar önemli ve bir o kadar zaman ayrılması gereken bir süreçtir.
- ✎ Testlerde test sonuçları yorumlanırken sadece testlerin geçip kalmasına değil aynı zamanda altta yatan sebeplerin araştırılmasına da önem gösterilmelidir.
- ✎ Bugün artık biliyoruz ki bir hata ne kadar geç tespit edilirse onu düzeltmenin maliyeti de katlanarak artar.
- ✎ Yazılım testi bu kadar önemli bir işlev olduğu için de mutlaka ayrı bir ekibin bu iş için görevlendirilmesi gereklidir.
- ✎ Yine tecrübelerimizle biliyoruz ki iyi yazılımcılar kendi kodları için genellikle iyi testçiler değildirler, bunun en önemli sebebi de insanın kendi hatalarını görememesidir, zaten kendi hatalarını görebilseydiler kodu yazarken doğru yazarlardı.

"Bir kodlayıcı kendi kodunu test etmek için uygun bir seçim değildir."

Weinberg Yasası

Test ve Birleştirme (Entegrasyon) Aşaması

- ☞ Testte dikkat edilecek konulardan bir diğeri de ürünün test ile kapsanan kod miktarıdır. Yani testin kodun ne kadarındaki hatalarını çıkarabildiğidir. Testin kalite unsurlarından biri de budur.
- ☞ Ancak yüzde yüz test ne yazık ki mümkün değildir. Yazılımın karmaşıklığı ve dallanıp budaklanma özelliğinden dolayı kodunuzu ne kadar test ederseniz test edin mutlaka hatalar kalacaktır. Bu sebeple testlerde öncelik ve kapsam belirlemek önemlidir.

"Bir programda ne kadar çok hata bulduysanız (artık hata kalmadı zannetmeyin) bir o kadar daha hata var demektir" :)

Anonim

"Testler yazılım hatalarının varlığını gösterir ancak ne yazık ki hataların yokluğunu gösteremez."

Edsger W. Dijkstra

Test ve Birleştirme (Entegrasyon) Aşaması

- ✎ Test sırasında dikkat edilmesi gereken bir diğer konu da testlerin sürekliliğidir. Genelde testleri ürün hazırlandıktan sonra yapma gibi bir hataya düşülür, oysa ki testler daha ürünün gereksinimleri hazırlanırken yapılmaya başlanmalıdır.
- ✎ Bunun mutlaka klasik anlamda test olması gerekli değildir, gereksinim gözden geçirmeleri de test sayılır.
- ✎ Daha sonra kod gözden geçirme toplantıları, otomatik kod tarayıcılarla kodun taranması, birim testleri, tümleştirme testleri, sistem testleri, kabul testleri, kurulum testleri, gerileme (regresyon) testleri gibi birçok testin ürünün geliştirilme süresi boyunca yapılması gereklidir.

"Test süreci hem işlevi hem de yapıyı test etmelidir."

Anonim

- ✎ Yazılım testleri konusundaki standartları listeleyecek olursak:
 - IEEE Standard of Software Test Documentation (IEEE/ANSI Standard 829),
 - IEEE Standard of Software Unit Testing (IEEE/ANSI Standard 1008),
 - IEEE Standard of Software Quality Assurance Plans' (IEEE/ANSI Standard 730).

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Bir Hata Sadece Bir Kez Bir İnsan Tarafından Tespit Edilmelidir

- Programlama metodunun önerilerinden biri de budur ve gereksiz iş tekrarını azaltmayı amaçlar, bir hata tespit edildiğinde derhal o hatayı belirleyen bir test yordamı yazılmalı ve otomatik test listesine eklenmelidir.
- Daha sonra böyle bir hata oluşursa otomatik testler bu hatayı hemen tespit edecek ve önceden not edilmiş olan çözüm uygulanacaktır. Bu sayede bu durumda bir yazılımcı günlerini bu hatayı tespit etmek ve düzeltmek için harcamayacaktır.
- Yararı çok olan bu tür testlere gerileme testleri (regresyon testleri) adını veririz. Bu testleri yazılımın gerilemesine izin vermeyen testlerdir anlamında düşünebilirsiniz.

"Bulunan her hata için ilişkili bir test yazılmalıdır. Bu test projenin ana test kütüphanesine kalıcı olarak eklenmelidir."

Bertrand Meyer – Eiffel Programlama Dilinin Yaratıcısı

Test ve Birleřtirme (Entegrasyon) Ařaması



Test Yapmanın Genel Kuralları

- **Erken Test Edin** - Sistemde çıkacak bir hatanın gereksinim aşamasında çıktığında bir birimlik düzeltme maliyeti varsa kodlama sırasında aynı hata bulunduğunda onlarca birim maliyetinin olduğunu ve ürün sahaya sürülüşünde aynı hatanın bulunması durumunda binlerce kat fazla düzeltme maliyetinin olduğunu biliyoruz. Bu sebeple testler mümkün olduğunca erken başlamalıdır. Gereksinimlerin toplanma aşamasında gözden geçirme toplantıları olarak başlayan aktiviteler de testlerin bir parçasıdır.
- **Sık Test Edin** - Her vazılım sürümü ve iterasyonundan sonra mutlaka testler yapılmalıdır. Gerileme (regresyon) testleri uygulanmalıdır. Gerileme testleri bir iterasyondan sonra yapılan yeni eklemelerin sistemi bozup bozmadığını tespit etmek için yapılan testlerdir. Testler projenin en başından en sonuna kadar biçim değiştirirler. En başta gereksinim gözden geçirmeleri, teorik hesapların sağlamaları ve kontrolleri şeklinde başlayan testler daha sonra klasik anlamdaki testler ile devam eder ve nihayet entegrasyon ve saha testleri ile devam eder.
- **Yeterince Test Edin** - Bir sistemi tamamen ve tüm yönleri ile test etmek gerçekçi bir yaklaşım değildir. Test etmenin bir maliyeti vardır. Testin getirisi maliyetinden yüksek olduğu sürece test yapılır. Yeterince test yapmak ne kadar önemli ve faydalı ise fazla test yapmak da o kadar maliyetlidir. Bu sebeple test getirisini maksimuma ulaştırmak için çeşitli optimizasyon yöntemleri uygulanır. Örneğin:
 - Sistemin en riskli alanları daha yoğun test edilir, öncelik sıraları uygulanır.
 - Yeterince test yazılır ve bu testler tekrar tekrar kullanılabilir şekilde düzenlenir.
 - Otomatik test yöntemleri kullanılır.
 - İstatistiki hesaplamalar yapılır. Örneğin başlangıçtaki tahmini hata oranları, testlerin hata bulma oranları ve testler sonuna kalan hataların tahmini miktarı gibi değerlerden faydalanılır.

Test ve Birleştirme (Entegrasyon) Aşaması

☞ Projede Yapılabilecek Testler Nelerdir?

- **Birim (Unit) testleri:** Yazılımı oluşturan her parçanın tek olarak testidir. Bu parçalar bir çalıştırılabilir program, bir dinamik kütüphane (dll), bir sınıf (class) hatta bir özellik grubu bile olabilir.
- **Birleştirme (Entegrasyon) testi:** Yazılımı oluşturan birimler bir araya getirildikten sonra uyumlu çalışıp çalışmadıklarını kontrol etmek için yapılır.
- **Performans testi:** Ürünün zorlu koşullarda sağlıklı çalışıp çalışmadığını saptamak için yapılır. Çeşitli tipleri mevcuttur:
 - **Mutlu patika (Happy path) testi:** Sistem zorlanmaz, sistemin beklediği en uygun değerler sisteme girdi olarak verilir. Genelde sistemin daha tamamlanmadığı, yeni yeni tümleştirilmeye çalışıldığı zamanlarda uygulanan yöntemdir. Daha sonra diğer testler ile desteklenmesi gerekir.
 - **Çevresel ortam testi:** Program değişik ortamlarda değişik makinelerle, değişik insanlar tarafından kullanılır, hatta ürünün özelliğine göre sıcaklık, titreşim, voltaj, vs değerler değiştirilerek ürün test edilir.
 - **Toparlama testi:** Sistemin yazılım/donanım çökmelerinde, elektrik amalarında, sıcaklık, nem, mekanik zorlama durumlarında veya diğer anormal durumlarda dayanıp dayanmayacağını ve bu durumlarda geri toparlanıp toparlanmayacağını, bunu başarabilme oranını ölçen bir testtir.
 - **Koşturma ortamı testi:** Yazılımın sadece laboratuvar sisteminde değil diğer birden fazla koşturma ortamında sağlıklı çalıştığını test etmek için yapılır.

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Projede Yapılabilecek Testler Nelerdir? (devam)

- **Uyumluluk testi:** Yazılımın çeşitli donanım, (uygunsa) işletim sistemi, ağ (network), yazılım ile uyumlu çalıştığının testidir.
- **Uç değerler testi:** Sistemin alabileceği maksimum ve minimum değerler sisteme girdi olarak verilir.
- **Hatalı kullanım testi:** Sisteme yanlış, eksik bilgiler girilir, yanlış zamanda yanlış komutlar verilir, bu ve benzeri durumlarda sistemin çökmediği test edilir.
- **Yükleme (Load) testi:** Sistemin sınırları zorlanır, örneğin kayıt tutuluyorsa maksimum kayıt miktarı yaratılır (veri yükleme), kullanıcı bağlanıyorsa maksimum kullanıcı ile bağlanılır, hafızası zorlanır, varsa sabit diski zorlanır, vs.
- **Rastgele değer testi:** Rastgele değerler girildiğinde sistemin doğru çalıştığı teyit edilmeye çalışılır. Diğer testlerin açıklarını kapatan bir sigorta gibidir. Özellikle uç değerler testinde düşünülmemiş olan durumları iyi saptar. Örneğin öngördüğümüz maksimum değer ve minimum değerlerde sistem sağlıklı çalışıyordur ama negatif bir değer verildiğinde sistem çöküyor olabilir, uç değer testinde bu durum göz önüne alınmamışsa rastgele testlerde bu durum yakalanabilir.
- **Kurulum/silinin testi:** Yazılımın kurulum (installation) ve kaldırma işleminin (uninstallati- on) sorunsuz olarak yapıldığının testidir. Yazılımı defalarca kurup silmek, değişik platformlarda, değişik zamanlarda kurup silmek, değişik kod büyüklüklerinde, konfigürasyonlarda veya değişik versiyonlarla bu işlemi tekrarlamak örnek olarak verilebilir. Yazılımı yükseltmek (upgrade) de diğer bir örnek olabilir.
- **Güvenlik testi:** Sistemin yetkili olmayan kişilerce kullanılmaya izin vermeme yeteneği, yazılımının ve verilerinin zarar görmeme yeteneği gibi testler yapılır

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Projede Yapılabilecek Testler Nelerdir? (devam)

- **Kullanım Senaryoları testi (işlev testi):** Ürünün değişik kullanım öykülerinde sağlıklı çalışıp çalışmadığı test edilir. Bu kullanım öykülerinin şartları proje gereksinimlerinde yazılı olabilir veya yazılması ihmal edilmiş olabilir, bu önemli değildir, zaten amaçlardan biri de gereksinimlerde yazılı olmayan ama ürünün kalitesi için gerekli olan özelliklerin varlığını test etmektir.
- **Kullanım kolaylığı testi:** Sistemin kullanıcı dostu olup olmadığı test edilir. Örneğin sistemde ekran ve tuş takımı varsa genel kabul görmüş kısa yollara uyulup uyulmadığı kontrol edilir. Yazılımcı ve testçiler zaten sistemi rahatça kullanabilecekleri için bu testte kullanılmaz. Muhtemel kullanıcılar, hatta bazen herhangi bir insan kullanılır.
- **Yoldan geçen adam testi:** Yazılımcılar ve testçiler sistemi normal kullanım şekline göre kullandıkları için bazen kritik hataları bulamazlar. Oysa herhangi bir insana sistemi kullandırdığınızda bazen sistemi hiç tahmin etmediğiniz şekilde çökertir. Bu sebeple bu tip teste yoldan geçen adam testi denir.
- **Kabul testi:** Gereksinimlerin doğru olarak gerçekleşip gerçekleşmediğini test etmek için yapılır.
- **Saha, kullanım veya piyasaya sunum testi:** Bu testlerde sistemin gerçekten çalışacağı ortamlarda veya sistemi gerçekten kullanacak kişilerin kullanımı ile testler yapılır.

Test ve Birleřtirme (Entegrasyon) Ařaması

☞ Projede Yapılabilecek Testler Nelerdir? (devam)

- **Gerileme (Regresyon) testi:** Ürüne yeni özellik eklendiğinde performans veya çalışma doğruluğunda bir gerileme olup olmadığını tespit etmek için yapılır.
- **Testlerin testi:** Ürünü test etmek için kullanılan testlerin doğruluğunu sorgulamak için yapılır. Bu testler dokümanlarda madde madde yazılı testler olabileceğı gibi test programları veya otomatik kořturulan testler olabilir. Örneğın otomatik testleri test etmek için en çok kullanılan yöntemlerden biri testlerden birisini bilinçli olarak kalacak şekilde ayarlamaktır, yani bu test her seferinde başarısız sonuçlanmalıdır. Eğer bu test başarılı sonuç verirse test ortamında gerçekten bir tutarsızlık olduğı anlaşılır. Aynı şekilde diğerk bir test de her zaman geçecek şekilde ayarlanır. Eğer bu test bile geçmezse test ortamında bir sorun olduğı sonucuna varılır.

"Yeni yazdığınız program sizin bilgisayarınızda sorunsuz çalışıyorsa bile o anki haliyle yandaki bilgisayarda da sorunsuz çalışma olasılığı çok düşüktür."

Hakan Atbař – Kitabın Yazarı

Test ve Birleştirme (Entegrasyon) Aşaması

∞ Testler kapsamlarına göre de sınıflandırılırlar:

- **Kara kutu (Black box) testi:** Kara kutu testi gereksinimlerin karşılanıp karşılanmadığını ve işlevi test eden bir test biçimidir. Sistemin iç detayları ile ilgilenmez. Belli girdilere karşılık istenilen sonucun çıkıp çıkmayacağı ile ilgilenir.
- **Beyaz kutu (White box) testi:** Kara kutu testinin tersine işlev yanısıra sistemin iç işleyişi ve yapısını da test eden testlerdir. Herhangi bir seviyede olabilirler ancak genel uygulamada birim testi seviyesinde yapılırlar. Kod kapsamı, dallanma kapsamı gibi yolları içerebilir.
- **Gri kutu (Gray box) testi:** Sistemin iç yapısı da göz önüne alınır ancak kara kutu testleri gibi yapılır. Kara kutu testlerinden farkı sisteme normal koşullarda verilemeyecek girdilerin de sisteme verilebilecek olmasıdır.

Test ve Birleştirme (Entegrasyon) Aşaması

- ∞ Testler yapılış metotlarına göre de sınıflandırılırlar:
- **Statik testler:** Bu testler klasik anlamda testler değildirler, statik testlerde kod koşturulmaz. Kod gözden geçirmeleri, gereksinim çelişme ve uyum belirlemeleri, otomatik programlarla veya elle söz dizimi (Syntax) kontrolü, anti-desen (anti-pattern) kontrolü bu kapsama girer.
 - **Dinamik testler:** Kod koşturularak yapılan testlerdir, sisteme belli girdiler verilerek sistemin davranışı gözlenir.

Test ve Birleştirme (Entegrasyon) Aşaması

Doğruluk Gösterimi (Verification) ve Geçerlilik Gösterimi (Validation)

- Bir yazılım üretilirken hem süreç boyunca kontroller yapılır hem de süreç sonunda testler yapılır.
- Ürünü doğru olarak üretmek için süreç boyunca yaptığınız kontroller doğruluk gösterimi (verifikasyon) alanına girer.
- Ürünün müşterinin istediği ürün olup olmadığını kontrol etmek ise geçerlilik gösterimi (validasyon) alanı içerisinde incelenir.
- Karıştırılan bu özellikler için aşağıdaki özlü söz faydalı olabilir:
 - Ürünü doğru üretip üretmediğinizi kontrol etmek verifikasyonun alanına girer.
 - Doğru ürünü üretip üretmediğinizi kontrol etmek validasyonun alanına girer.
- Kimi zaman verifikasyonun kalite güvence faaliyetleri ve validasyonun ise kalite kontrol faaliyetleri olarak tanımlandığı da olur.

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Projeye Başlarken Testlere de Başlayın, Projenin Bitmesini Beklemeyin

- Yazılımda bulunacak hataların maliyet eğrisini incelediğimizde projeye başlarken bulunan hataların düzeltme maliyeti 1 birim iken aynı hata ancak proje sonu testleri sırasında bulunabilirse bu maliyet 10-100 birime çıkmakta ve proje sahaya sürüldükten sonra bulunacak hatayı düzeltme maliyeti 50-1000 kadar çıkabilmektedir.
- Peki ne yapalım. Projeye başlarken, daha ortada yazılmış kod yokken mi teste başlayalım. Cevabımız: başlayın, hemen başlayın.”
- Peki nasıl?

Test ve Birleştirme (Entegrasyon) Aşaması

☞ Projeye Başlarken Testlere de Başlayın, Projenin Bitmesini Beklemeyin

- Gereksinimlerin belirlenmesi sırasında müşteri ve ekibiniz arasında gözden geçirme toplantıları yapın. Yazılımlardaki hataların çoğu sanıldığı gibi kodlayıcının yaptığı hatalar değil gereksinimlerin belirlenmesi sırasında yapılan hatalardır.
- Tasarıma zaman ayırın ve tasarım gözden geçirme toplantıları yapın. Bir öneriye göre zamanın %40'ı gereksinimlerin belirlenmesi ve tasarım için harcanmak, %20 si kodlama için harcanmalı diğer %40'ı ise test ve hata düzeltimi için harcanmalıdır. Bu değerler her proje ve her organizasyon için değişebileceğinden sadece tasarımın ne kadar önemli olduğunu vurgulamak için belirtiyoruz.
- Kod yazımı esnasında mutlaka kod gözden geçirme toplantıları yapın. Yazılım gurupları test sırasında bulunan hatalar kadar hatanın bu toplantılarda belirlenebildiğini söylüyorlar. Üstelik test sırasında tasarımdaki sorunlar bulunamazken bu toplantılarda tasarıma ait sorunlar da belirlenebiliyor.

"Eğer onu test edemiyorsanız onu üretmeyin (bile). Eğer onu test etmiyorsanız atın gitsin."

Test ve Birleřtirme (Entegrasyon) Ařaması

∞ Projelerde Test Ekibi veya Elemanı alıřtırmak

- Yazılım projelerinde test yapacak adam alıřtıralım mı?” diye sorarsanız. Bu soruya cevabımız büyük bir **“Evet”** olur. Hatta mümkünse ayrı bir test ekibinin olması gereklidir.
- “Peki yazılımcılardan bazılarını test elemanı olarak ayıralım mı?” diye sorarsanız cevabımız řudur; “Genellikle yazılımcılardan pek iyi testi olmaz”. Test işi de bir kariyerdır. Bunu sakın unutmayın. Test etme yeteneğine sahip bazı insanlar vardır ki diğerk testi ve yazılımcılardan kat ve kat daha fazla sayıda ve kritik hataları tespit edebilirler.
- Yıllardan beridir yazılım yapan kişiler bile hala “Biz yazılımımızı doğru yazıyoruz, testi alıřtırmaya gerek kalmıyor.” diyebiliyorlar. Oysa ki hatasız yazılım diye bir řey yoktur. Her yazılımın az ve ya ok mutlaka hataları vardır.

"Müşterilerinizi hatalarınızı bulan, üstelik bedava alıřan test elemanınız olarak kullanmayın. Bu size prestij ve para kaybettirir."

Test ve Birleştirme (Entegrasyon) Aşaması

∞ Otomatik Testler ve Kullanımları

- Bir yazılımın testleri insan tarafından veya otomatik bir sistem, program tarafından yapılabilir. Otomatik testlerin yararları şunlardır:
 - **Güvenilirlik:** Otomatik testlerde doğru olduğu kanıtlanmış bir test her zaman koşturulabilir, yani insan hatası şansı pek yoktur.
 - **Tekrarlanabilirlik:** Otomatik testler istenildiği zaman istenildiği kadar tekrar edilebilir.
 - **Programlanabilirlik ve Dakiklik:** Otomatik testlerde istenilen test adımı istenilen anda yapılabilir, zamanlaması iyidir.
 - **Kapsamlılık:** Otomatik testler ile test kapsamını ayarlamak kolaydır.
 - **Kalitelilik:** Otomatik testler yazılı olduğu için toplam kalite etkenidir.
 - **Hızlılık:** Otomatik testler el ile yapılan testlerden daha hızlıdır.
 - **Düşük Maliyet:** Otomatik testlerin yazılması maliyetlidir ancak testleri kullanması neredeyse maliyetsizdir.

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Otomatik Testler ve Kullanımları (devam)

- Otomatik test ne zaman yapılır:
 - Herhangi bir değişiklik sonunda tekrar test yapmak maliyetli olduğunda (uzun sürmek vesaire)
 - Herhangi bir değişiklik sonunda sisteme girecek bir hatanın maliyeti yüksek olduğunda
 - Tam kod kapsamı gerektiğinde
 - Kritik sistemlerde insan hatasına tahammül olmadığında
 - İnsan kullanılarak yapılması pratik olarak zor veya imkânsız olan testlerde, örneğin bir sunucuya binlerce kişinin aynı anda bağlanmasını test etmek için
 - Manuel olarak yapan insanın psikolojisini bozan tekrarlı, monoton testlerde,
 - Her sürümden sonra yapılması düşünülen gerileme (regresyon) testlerinde. Zaten gerileme testleri otomatik testlerin en verimli olduğu alandır.

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Otomatik Testler ve Kullanımları (devam)

○ Otomatik test ne zaman yapılmaz:

- Grafik Kullanıcı Arabirimi (GUI) gibi sürekli ufak tefek değişikliklerin olduğu ve aslında herhangi bir hata olmadığı halde testlerin sürekli yanlış sonuç vereceği durumlarda,
- Test yazımının maliyetinin organizasyonun kaldırabileceğinden fazla olduğu durumlarda, veya kar-zarar oranının düşük olduğu durumlarda, örneğin sadece bir yıl daha kullanılacak eski bir yazılım için otomatik testler yazmaya girişmek mantıklı olmayabilir,
- İnsan kullanımının gerekli olduğu yerlerde, örneğin web sayfası kullanım öyküsü testi otomatik yapılmaz, insan gereklidir,
- Otomatik test yazımının ürünün geliştirme hızını yakalayamayacağı, örneğin bir kaç aylık planı olan, iki üç kişi ile yazılan ve bir kez kullanılacak, acil geliştirilmesi gereken bir yazılım için otomatik test geliştirmek pek de pratik değildir,
- Kalitesi çok kötü olan yazılımlarda otomatik testleri yapmak da sonucu yorumlamak da bir dert olabilir, bu durumda deneyimli testçinin kendi yöntemlerine güvenmekten başka bir çaresi kalmaz.
- Otomatik testlere teknik olarak imkân olmadığına, örneğin gömülü özel bir sistemde otomatik test çok zor olabilir, bir grafik programında zor olabilir, üçüncü el bileşenler kullanan bir programda testi koşturacak arabirim eksikliğinden dolayı otomatik test imkânsız olabilir.

Test ve Birleştirme (Entegrasyon) Aşaması

∞ Tümlleştirme (Entegrasyon) ve Yazılımdaki Büyük Patlama (Big Bang) Teorisi

- Diyelim ki yazılımınızı güzelce tasarladınız ve minik kolayca gerçekleştirilebilir parçalara ayırdınız. Daha sonra da bu yazılım parçalarını kodladınız ve birim testlerini yaptınız. İşte şu anda çok dikkat etmeniz gereken bir safhadasınız; **tümlleştirme** safhası.
- Yazılım sürecinde en sık yapılan hatalardan biri yazılımı, parçalarının yazılıp bir araya getirildiğinde hiç sorun olmadan çalışacağını düşünmektir. Hatta projelerde bu tümlleştirme safhası için süre bile ayrılmadığı olur. Oysa ki parçaları ne kadar dikkatli kodlarsanız kodlayın ve istediğiniz kadar birim testi yapmış olun tümlleştirme safhası sırasında sorunlar çıkacaktır
- Tümlleştirme safhasında tüm modülleri birdenbire bir araya getirip bir arada çalışmalarını bekleyerek kocaman bir test yaptığınızda evrenin başlangıcındakine benzer bir patlama yaşarsınız, bu yüzden bu yanlış yaklaşıma Big Bang yaklaşımı denilir.
- İşin doğrusu bir plan dahilinde kontrollü olarak parçalar, bir araya getirmektir.
- Her parça eklendiğinde testler yapılmalı ve ancak testler geçtiğinde diğer bir parça eklenmelidir.
- Bu tümlleştirme safhası için mutlaka yeterince süre ayrılmalıdır.

Test ve Birleştirme (Entegrasyon) Aşaması

☞ Yazılım Testinin Bazı İlkeleri

- Yazılım testi konusundaki kurallar yeni yeni geliştirilmeye başlanmıştır.
- Şimdilerde popüler olan iki adet liste vardır.
 - Bertrand Meyer'den "Testin Yedi İlkesi",
 - Diğer ustaların söylediklerinden derlenmiş olan "Testin Yedi İlkesi".

Test ve Birleştirme (Entegrasyon) Aşaması

∞ Yazılım Testinin Bazı İlkeleri

- Bertrand Meyer'den "Testin Yedi İlkesi"
 - **İlke 1: Tanım ilkesi** - Bir programı test etmek, o programın hata yapmasını (çakmasını) sağlamaktır.
 - **İlke 2: Testler ve spesifikasyonlar** - Testler spesifikasyonların yerine geçemez.
 - **İlke 3: (Gerileme) (Regression) Testleri** - Bulunan her hata için ilişkili bir test yazılmalıdır. Bu test projenin ana test kütüphanesine kalıcı olarak eklenmelidir.
 - **İlke 4: Test geçti-kaldı kriteri olarak kontratlar.** - Test-geçti-kaldı kriterleri bir programın kontrat olarak parçası olmalıdır. Testin geçmesi veya kalması, kontratın sağlandığının kontrolünü içeren otomatik bir süreç haline getirilmelidir.
 - **İlke 5: Manüel ve otomatik testler** - Etkili bir test süreci hem manuel hem de otomatik olarak hazırlanmış testleri içermelidir.
 - **İlke 6: Test stratejilerinin deneysel değerlendirilmesi** - Her test stratejisini açık ve net kriterler ile ve tekrar edilebilir test süreci ile objektif olarak değerlendirin.
 - **İlke 7: Değerlendirme kriteri** - Bir test stratejisinin en önemli özelliği, zamanın fonksiyonu olarak ortaya çıkarabildiği hata sayısıdır.

Test ve Birleştirme (Entegrasyon) Aşaması

🌀 Yazılım Testinin Bazı İlkeleri

○ Ustalardan derlenmiş “Testin Yedi İlkesi”

- **İlke 1:** Test yazılım hatalarının varlığını gösterir ancak yokluğunu gösteremez. (Edsger W. Dijkstra)
- **İlke 2:** Her şeyi test etmek imkânsızdır. (Zamanınız ve kaynaklarınız her şeyi test etmeye yeterli değildir, risk analizi yaparak testleri makul seviyede tutmalısınız.)
- **İlke 3:** Erken test edin. (Test hazırlığı proje başında başlamalıdır, en sona bırakılmamalıdır.)
- **İlke 4:** Hataların Kümelenmesi - Hataların büyük bir kısmı belli yazılım modüllerinden çıkar. (Hataların çoğu veya önemlileri belli birkaç modülde bulunur.)
- **İlke 5:** Tarım İlacı Paradoksu (Pesticide paradox): Testler kullanıldıkça daha az hata tespit etmeye başlarlar. (Özellikle otomatik testler kullanıldıkça ilgili hatalar düzeltileceği için artık pek fazla hata tespit edemezler. Test kütüphanenize zaman zaman yeni testler de eklemelisiniz.)
- **İlke 6:** Test konuya göre hazırlanmalıdır. (Testler yazılım modülünün içeriğine, kullanım alanına, kullanım şartlarına dikkat edilerek hazırlanmalıdır. Örneğin evde kullanılacak bir cihaz için sıcaklık testleri çok önemli değilken askeri cihazlar için sıcaklık ve ortam testleri en önemli testlerdendir.)
- **İlke 7:** Üründe hataların bulunmaması ürünün gereksinimleri karşıladığı ve kaliteli olduğu anlamına gelmez.

Sürüm ve Teslim Aşaması

- ❧ Organizasyonunuzdaki birim testler başarı ile bittikten, tüm bileşenler birleştirildikten ve son testler de yapıldıktan sonra artık farklı bir süreç başlar.
- ❧ Bu süreç ürünün niteliğine göre bireysel ürünler için piyasaya sürüm safhası, organizasyonel ürünler için de teslim aşamasıdır.
- ❧ Bireysel ürünler için pazarlama, satış gibi teknik olmayan faaliyetler de ağırlık kazanır. Organizasyonel ürünler için ise hala teknik zorluklar ana risktir.
- ❧ Bu safhanın en büyük özelliği bu safhanın çoğu zaman proje takvimlerinde yer alandan daha fazla sürmesi ve hiç akla gelmeyecek riskleri barındırmasıdır

Sürüm ve Teslim Aşaması

∞ Kurulum, Devreye Alım ve Saha Testleri İçin Kaynak ve Süre Ayırın

- Entegrasyon yani birimleri birleştirme testleri bittikten sonra çoğu organizasyon proje bitti zanneder. Hatta bazı anlaşmalarda kurulum, devreye alım ve saha testleri düşünülmemiştir bile.
- Oysa bu süre proje süresi içinde hatırı sayılır bir yer tutar. Müşteri memnuniyetini belirleyen aktivite de budur, çünkü müşteri sizin ne kadar özveri ile kod yazdığınızdan, gece gündüz mesai yaptığınızdan, süper bir plan yaptığınızdan haberdar değildir.
- Müşterinin gerçekten göreceği çalışmanız ve size vereceği kalite puanı sizin bu süre içinde yaptığınız çalışmanın kalitesine ve verimliliğine bağlıdır.
- Müşteri memnuniyetini etkileyen iki önemli faktörden biri ekran düzeni ve kullanımı ise diğer ana faktör de kurulum ve devreye alma aşamasının kalitesidir.

Sürüm ve Teslim Aşaması

🌀 Sahada Hata Ayıklayabilmenin (Debug Edebilmenin) Güzelliği

- Sahada hata ayıklayabilmek ihtiyaç olduğunda hayat kurtaran bir özelliktir.
- Sahada hata ayıklayabilmek için gerekli konfigürasyonu mutlaka hazırlayınız.
- Printf'ler ile veya mesaj kutuları ile hata ayıklamaya çalışmak saha stresi altında hataları bulmak değil yeni hataları kodunuza eklemekle sonuçlanabilir.
- Bu konuda size şunları önerebiliriz:
 - Eğer sahadaki sisteminiz gömülü bir sistem ise bir emülatör edinin,
 - Eğer sahadaki sisteminiz pahalı ve büyük bir sistem ise biraz daha masraf edip lisans alarak geliştirme ortamınızı sahadaki bu sisteme de kurun,
 - Eğer sahadaki sisteminiz ucuz ve küçük bir sistem ise kendinize hata ayıklama imkânları olan yedek bir test sistemi kurun.

Sonuçları Değerlendirme Aşaması

- ✎ Bu aşama çoğu zaman atlanan fakat gerçekten önemli bir aşamadır.
- ✎ Bu aşamada projenin genel bir değerlendirmesi yapılır.
- ✎ Proje sırasında yapılan yararlı uygulamalar, yapılan hatalar ve bunun gibi birçok şey masaya yatırılır ve incelenir.
- ✎ Unutmayınız ki başarının sırları şudur:
 - İyi bir planlama ve hazırlık evresi
 - Çok çalışma
 - Hatalardan ders alma

"Tecrübenin faydası olur mu? Hayır, (benzer) yanlışları yapmaya devam ediyorsak olmaz."

W. Edwards Deming – Toplam Kalite Yönetimi Felsefesinin Yaratıcısı

Sonuçları Değerlendirme Aşaması



Proje Sonunda Değerlendirme Toplantısı Yapınız. Peki Bu Toplantı Nasıl Yapılır?

- Her ekibin en değerli bilgisi kendi edindiği tecrübelerdir. Proje sonu değerlendirme toplantıları bu tecrübeleri oluşturmak ve ileriki projelerde daha başarılı olmak için en uygun yollardan biridir.
- Birçok organizasyon bir kere değil defalarca aynı hataları yapmaktadır. Bu sebeple proje sonunda bir değerlendirme toplantısı yapmak çok faydalıdır.
- Proje değerlendirme toplantısından sonra “Proje Sonrası Gözden Geçirme Raporu” hazırlanmalı ve yazılmalıdır. Bu rapordaki bir hata kaydı, kişilerin hatalarının kaydı niteliğinde olmamalıdır. Bu rapor ilerleme, düzeltme amaçlı bir rapor olmalıdır.
- Proje bittiği için bazen ekip üyeleri nasıl olsa proje bitti diye görüş bildirmekten çekinir veya üşenirler, bu durumu engellemek için değerlendirme toplantıları daha proje devam ederken, projenin her ana basamağı sonunda yapılabilir.
- Proje sonu toplantısından sonra yapılan önemli bir hata raporun hazırlanıp rafa kaldırılmasıdır. Oysa çalışma gruplarına işlem maddeleri açılmalı ve raporda sözü edilen geliştirici ve düzeltici işlemler mutlaka uygulanmalıdır.

"İnsanları değil iş akışını düzeltin."

Anonim

Bakım, Destek ve İşletim Aşaması

- ✎ Bakım aşaması genellikle ürün sahadayken çıkabilecek yazılım hatalarını düzeltmek için yapılan faaliyetler olarak tanımlanır.
- ✎ Ancak pratikte bu böyle olmaz. Kullanıcılar ürünü kullandıkça ürünlerdeki eksiklikleri, ürünle ilgili isteklerini de raporlarlar ve bu yeni özellikler de bakım faaliyetleri sırasında araya ilıştırılmak zorunda kalınır.
- ✎ Ayrıca yazılım hatası olmayan ancak kullanıcının anlamakta güçlük çektiği özelliklerin kullanıcıya tekrar tekrar açıklanması ve bunun gibi hizmet işleri de bakım aşaması içerisine girerir.
- ✎ Bakım konusunda yapılan en büyük yanlışlardan biri programın son halini aldığı ve artık bakım ihtiyacı olmayacağıdır.
- ✎ Bakım aşaması genellikle ufak bir işmiş gibi değerlendirilir. Oysa yukarıda da bahsettiğimiz gibi bakım aşamasının göze görünmeyen birçok iş kalemi olduğu için aslında projedeki en yüksek maliyetli aşamalardan biridir.

"Ve şöyle dedi büyük usta...

Bir program üç satır bile olsa, onun bakımı bir gün mutlaka zorunlu olacak. "

Geoffrey James, The Tao Of Programming kitabından.

Bakım, Destek ve İşletim Aşaması

🌀 Bakım Faaliyetleri

Bakım faaliyetleri birkaç dala ayrılır:

- **Önleyici bakım faaliyetleri:** Oluşan bir sorunun bir daha olmaması için veya yeni fark edilen bir riskin kötü sonuçlar doğurmaması için kodda yapılan değişikliklerdir.
- **Düzeltilici bakım faaliyetleri:** Ortaya çıkan hataların düzeltilmesi için yapılan faaliyetlerdir.
- **Adapte edici bakım faaliyetleri:** Çevresel etmenlerin değişimi ile (örneğin çalışma koşullarının değişmesi, teknolojinin ilerlemesi, vb) yeni koşullara kodun ayak uydurması için yapılan faaliyetlerdir.
- **İlerletici bakım faaliyetleri:** Yazılıma yeni özellikler eklenmesi, müşterinin yeni isteklerinin koda eklenmesi için yapılan faaliyetlerdir.

"Bir insanın berbat yazılımı, diğer bir insan için tam zamanlı bir iş imkânıdır."

Jessica Gaston